RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)

SEMESTER V

SI LAB

PROJECT REPORT

STATISTICAL ANALYSIS AND REGRESSION-BASED PREDICTION OF DAILY TEMPERATURE PATTERNS

Submitted by

Student Name	Roll No.	Enrollment No.
Swara Pande	B1-B2-19	**[Enter Enrollment No.]**

Academic Year: 2026–27

Statistical Analysis and Regression-Based Prediction of Daily Temperature Patterns

SI Lab Project Report

Swara Pande | Roll No. B1-B2-19 | Semester V, B.Tech. CSE (AI & DS)

1. PROJECT OBJECTIVE

The purpose of this project is to study a full year of daily weather observations (365 days of 2025) using the statistical methods covered in the SI Lab and to build a machine-learning model that predicts the daily temperature from the other weather conditions. The project asks two connected questions: (i) what do the data say about how daily temperature is distributed and what it depends on, and (ii) how well can that temperature be predicted for days the model has not seen. The expected output is a set of statistical findings, a comparison of five regression models, one selected and saved model, and a small GUI in which a user enters weather conditions and receives a predicted temperature.

The temperature is a continuous quantity, so this is a regression problem rather than a classification problem. The objectives of the suggested classification template have therefore been adapted: classification metrics (accuracy, precision, recall, F1) are replaced by regression metrics (MAE, RMSE, R^2), and the confusion matrix is replaced by an actual-versus-predicted plot. The objectives are:

- Load and explore the Daily Temperature Patterns dataset and summarise it with descriptive statistics.
- Compare the temperature distribution with the Normal, Uniform and Binomial distributions and measure its skewness.
- Draw samples by simple random, systematic, stratified and cluster sampling and compare them with the population.
- Demonstrate the Central Limit Theorem, bootstrap resampling, estimator behaviour (mean versus median) and 95% confidence intervals with coverage checks.
- Perform hypothesis tests (one-sample t and Z, chi-square, Pearson/Spearman correlation, two-way ANOVA) and covariance analysis.
- Preprocess the data and split it chronologically into training and testing sets.
- Train and compare multiple regression models (Linear, Ridge, Lasso, Decision Tree, Random Forest).
- Select the best model using time-series cross-validation, then save and reload it.
- Develop a GUI that accepts weather inputs and displays the predicted temperature.

2. INTRODUCTION

Temperature is the most commonly reported weather variable and affects agriculture, energy demand, health and daily planning. Daily temperature is shaped by the season, by cloud cover and sunshine, by humidity and by rainfall, and these variables are themselves related to each other. Understanding which relationships are real, and how reliably temperature can be

predicted from the others, is a good setting for practising statistical inference and supervised learning together.

The dataset used here records 13 measurements for each of the 365 days of 2025, including temperature, feels-like temperature, humidity, rainfall, wind speed, pressure, cloud cover, sunshine hours, season and day type. Because the data are a complete daily record, the project treats the full year as the population and draws samples from it, which makes it possible to see directly how far each sampling method or estimator lies from the true value. The same data are then used to train regression models. An interactive dashboard (Daily_Temperature_Patterns.html) was also prepared from the dataset to present the main findings visually.

2.1 Problem Statement

Given daily observations of humidity, rainfall, wind speed, pressure, cloud cover, sunshine hours, season and day type, analyse how daily temperature behaves statistically and build a regression model that predicts the temperature (°C) of a day, evaluated on later days that were not used for training.

2.2 Scope of the Project

Included: descriptive statistics, distribution comparison, four sampling techniques, sampling-distribution and confidence-interval experiments, hypothesis testing, covariance and correlation analysis, five regression models with hyperparameter search, model saving and loading, and a notebook-based GUI for prediction.

Limitations: the data cover a single year at a single (unnamed) location, so seasonal behaviour is seen only once. The models use only same-day weather variables and do not forecast future days. The GUI runs inside the Colab/Jupyter notebook using ipywidgets and is not a standalone desktop or web application.

3. PROJECT METHODOLOGY

The work followed these stages: data loading and exploration; cleaning and type conversion; statistical analysis (descriptive, sampling, inference, relationships); feature preparation and chronological splitting; training and tuning of regression models; evaluation and model selection; saving the selected model and wrapping it in a GUI.

3.1 Dataset Loading and Exploration

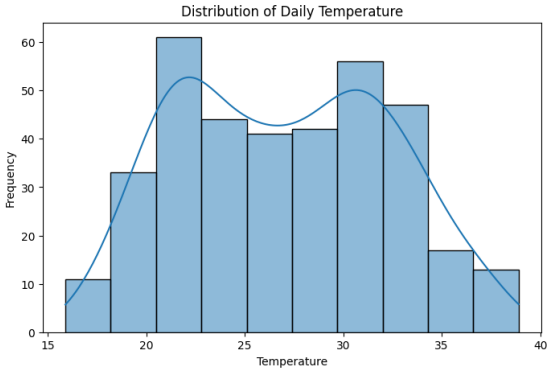
- **Dataset:** Daily Temperature Patterns (daily_temperature_patterns.csv), loaded with pandas into a DataFrame.
- **Size:** 365 rows and 13 columns, one row per day from 1 January to 31 December 2025.
- **Features:** Date, Temperature, Feels_Like_Temperature, Humidity, Rainfall_mm, Wind_Speed_kmph, Pressure_hPa, Cloud_Cover_Percent, Sunshine_Hours, Temperature_Change, Season, Day_Type and Year.
- **Target variable:** Temperature (°C), a continuous variable.

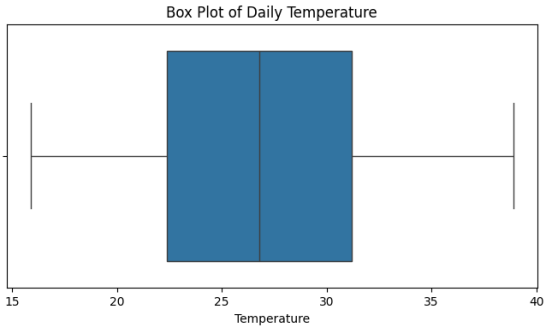
- **Data types:** Date is converted to datetime; the nine measurement columns are numeric (float); Season and Day_Type are categorical text columns.
- **Missing values:** Only one value is missing, Temperature_Change on 1 January 2025, because there is no previous day to compare with. No other column has missing values.
- **Duplicates:** 0 duplicate rows were found in the 365 records.
- **Class balance of categories:** Season (as calculated from the month) has 122 Monsoon, 92 Summer, 90 Winter and 61 Post-Monsoon days; Day_Type has 261 weekdays and 104 weekend days.

Table 1 summarises the temperature variable. The mean (26.98 °C) and median (26.8 °C) are almost equal and the skewness is 0.10, so the distribution is approximately symmetric. The histogram, however, shows two humps (near 22 °C and 31 °C) rather than a single bell shape, which reflects the cooler and hotter parts of the year.

Table 1: Descriptive statistics of daily temperature (°C)

Statistic	Value
Count	365
Mean	26.984
Median	26.800
Variance	28.352
Standard deviation	5.325
Skewness	0.101
Minimum	15.9
25th percentile	22.4
75th percentile	31.2
Maximum	38.9





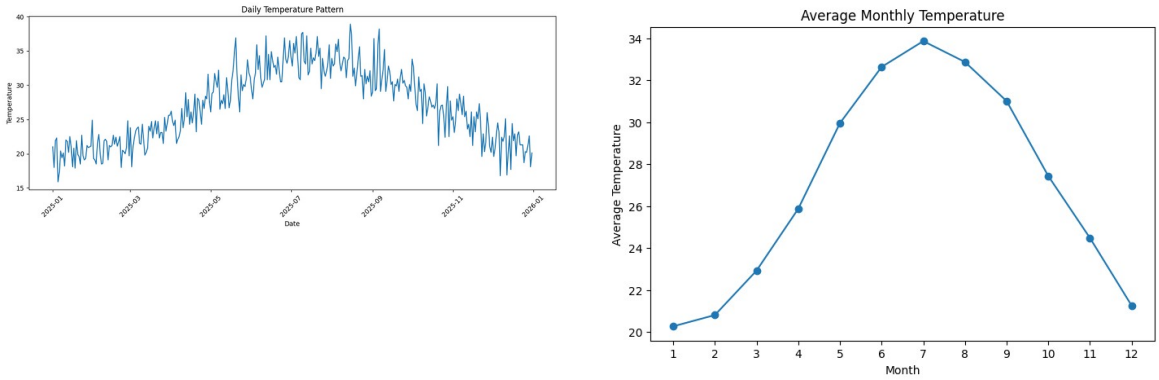


Figure 1: Exploratory plots: histogram with KDE, box plot, daily temperature through 2025, and average monthly temperature.

The box plot shows no outliers, and the time series shows a clear annual cycle. Average temperature by season (Table 2 and Figure 6) is highest in the Monsoon months and lowest in Winter. The season definition used throughout is: Winter (Dec–Feb), Summer (Mar–May), Monsoon (Jun–Sep) and Post-Monsoon (Oct–Nov).

Table 2: Mean temperature by season (°C)

Season	Days	Mean temperature
Winter	90	20.78
Post-Monsoon	61	25.98
Summer	92	26.26
Monsoon	122	32.61

3.2 Data Preprocessing

- Dates were parsed with pd.to_datetime and numeric columns were converted with pd.to_numeric (errors coerced to missing).
- Rows with a missing Date or Temperature were dropped; no such rows existed, so all 365 rows were kept. The single missing Temperature_Change value was not used by the models.
- A Season_Calculated column was derived from the month of the date and used for sampling, ANOVA and modelling.
- Categorical variables (Season_Calculated, Day_Type) are imputed with the most frequent value and one-hot encoded inside the model pipeline.
- Numeric variables are imputed with the median and standardised with StandardScaler, also inside the pipeline, so that the scaling is learned from training data only.
- The input features X and target y (Temperature) were separated, and the data were split chronologically: the first 80% (292 days, 1 Jan to 19 Oct 2025) for training and the last 20% (73 days, 20 Oct to 31 Dec 2025) for testing. A chronological split was chosen to avoid training on days that come after the test days.

3.3 Feature Extraction / Selection

No separate feature extraction was needed. Eight input features were used: six numeric (Humidity, Rainfall_mm, Wind_Speed_kmph, Pressure_hPa, Cloud_Cover_Percent, Sunshine_Hours) and two categorical (Season_Calculated, Day_Type). Feels_Like_Temperature and Temperature_Change were excluded deliberately: both are computed from or closely tied to the temperature itself (their covariance with temperature is 28.08 and nearly equals the variance of temperature, 28.35), so using them would leak the answer into the inputs.

Feature relevance was examined statistically before modelling (Section 6). Temperature is moderately correlated with humidity ($r = -0.34$) and rainfall ($r = 0.38$), is not significantly correlated with sunshine hours ($r = -0.04$, $p = 0.455$) or wind speed ($r \approx 0.00$, $p = 0.996$), and differs strongly between seasons. All six numeric features and the two categorical features were retained and regularised models (Ridge, Lasso) were used to control the influence of weak predictors.

3.4 Training Multiple Regression Models

Five regression models were implemented as scikit-learn pipelines (preprocessing followed by the regressor).

Sr. No.	Model Name	Purpose / Notes
1	Multiple Linear Regression	Baseline model that fits a linear combination of the standardised inputs by least squares.
2	Ridge Regression	Linear model with an L2 penalty that shrinks coefficients; reduces variance when predictors are correlated. alpha tuned over {0.1, 1, 10, 100}.
3	Lasso Regression	Linear model with an L1 penalty that can shrink weak coefficients to zero (built-in feature selection). alpha tuned over {0.01, 0.1, 1.0}.
4	Decision Tree Regressor	Non-linear model that splits the data by rules; max_depth {3, 5, 8} and min_samples_leaf {2, 5} tuned.
5	Random Forest Regressor	Ensemble of 100 trees that averages many trees to reduce variance; max_depth {4, 8, None} and min_samples_leaf {2, 5} tuned.

3.5 Performance Evaluation and Model Comparison

Because the target is continuous, the models are evaluated with Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and the coefficient of determination (R^2) on the 73 unseen test days. For selection, each model was also tuned by grid search with 5-fold time-series cross-validation (TimeSeriesSplit) on the training days, scored by RMSE. Lower MAE and RMSE and higher R^2 are better; R^2 below zero means the model predicts worse than simply using the mean of the training temperatures.

Table 3: Model comparison (tuned models; sorted by cross-validation RMSE)

Model	CV RMSE	Test RMSE	Test MAE	Test R^2
Ridge (alpha = 0.1)	4.6459	3.6219	2.8518	−0.4508
Linear Regression	4.6460	3.6271	2.8576	−0.4550
Decision Tree	4.8812	2.9810	2.2569	0.0172
Random Forest	5.0155	3.0665	2.4084	−0.0400
Lasso	5.1402	2.8458	2.2176	0.1043

The earlier untuned runs with fixed settings gave similar results: Linear Regression (MAE 2.8576, RMSE 3.6271, R^2 −0.4550), Ridge with alpha = 1 (2.8050, 3.5778, −0.4157), Lasso with alpha = 0.1 (2.2176, 2.8458, 0.1043), Decision Tree with depth 5 (2.2896, 2.9331, 0.0485) and Random Forest with depth 8 (2.5321, 3.2074, −0.1377).

3.6 Best Model Selection

The model selected by the notebook is Ridge Regression with alpha = 0.1, which has the lowest time-series cross-validation RMSE (4.6459). The selection rule uses only the training period, so the test days remain an unbiased check. The choice is justified as follows: Ridge and Linear Regression are almost tied on cross-validation, Ridge is slightly better and is more stable when predictors are correlated, and the models with the best test scores (Lasso, Decision Tree) were not better in cross-validation, so their test advantage cannot be relied on from one 73-day test period.

The selected model was saved to best_temperature_model.joblib with joblib, reloaded, and verified to give identical predictions to the original (check returned True). An important caveat, discussed in Section 6.2, is that on the test period every model has a high error compared with the spread of the data, so the selected model should be regarded as the best available of the tested models and not as a highly accurate predictor.

4. PROJECT IMPLEMENTATION

4.1 Tools and Technologies

Tool / Technology	Version	Purpose
Python	3.x (Colab runtime)	Programming language
Google Colab / Jupyter	Colab hosted runtime	Development environment
Pandas / NumPy	Colab default	Data handling and numerical computing
SciPy / Statsmodels	Colab default	Statistical tests, distributions, ANOVA

Tool / Technology	Version	Purpose
Scikit-learn	Colab default	Preprocessing pipelines, models, cross-validation, metrics
Matplotlib / Seaborn	Colab default	Visualisation
Joblib	Colab default	Saving and loading the trained model
ipywidgets	Colab default	GUI (input fields, dropdowns, button)
Chart.js (HTML dashboard)	4.4.1	Interactive dashboard of the findings

4.2 Module-wise Implementation

- **Dataset loading and exploration module:** reads the CSV, converts types, checks missing values, and plots the distribution, box plot and time series.
- **Statistical analysis module:** descriptive statistics; Normal/Uniform/Binomial comparison; simple random, systematic, stratified and cluster sampling; Central Limit Theorem and bootstrap experiments; mean-versus-median estimators; confidence intervals and coverage; hypothesis tests; covariance, correlation and two-way ANOVA.
- **Data preprocessing module:** median and most-frequent imputation, standardisation and one-hot encoding in a ColumnTransformer; chronological 80/20 split.
- **Feature selection module:** the eight input features are defined and leakage columns are excluded (no separate extraction step).
- **Model training and evaluation module:** five regressors trained, tuned by GridSearchCV with TimeSeriesSplit, and evaluated with MAE, RMSE and R^2.
- **Best model saving/loading module:** the selected pipeline is saved and reloaded with joblib and verified.
- **GUI input and prediction module:** ipywidgets form that builds a one-row DataFrame from the inputs and calls the loaded model.

4.3 GUI Implementation

The GUI is built with ipywidgets and displayed in the notebook. It contains six FloatText fields for the numeric weather inputs (Humidity, Rainfall_mm, Wind_Speed_kmph, Pressure_hPa, Cloud_Cover_Percent, Sunshine_Hours), two Dropdown menus (Season_Calculated and Day_Type) and a “Predict Temperature” button. The numeric fields start at the median of the training data (for example humidity 65.45) so that the form always holds a valid example, and the dropdown options are taken from the values present in the training data.

When the button is pressed, the values are collected into a single-row DataFrame with the same columns as the training features and passed to the loaded model. Because the saved object is the whole pipeline, the imputation, scaling and encoding are applied automatically, and the predicted temperature is printed below the button with two decimals, in the same unit as the Temperature column (°C). With the default inputs (the training medians, season Monsoon and day type Weekday), the saved Ridge model predicts about 32.14 °C. Figure 2 shows the layout of the interface.

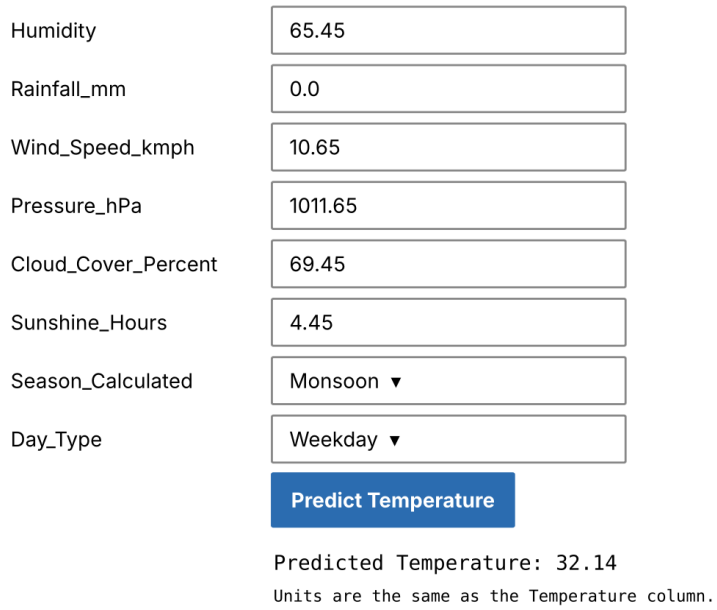


Figure 2: Layout of the prediction GUI with its default inputs and the prediction returned by the selected model (schematic rendering of the ipywidgets form).

5. COMPLETE CODE

The complete, runnable source code of the project is reproduced below in the order in which it appears in the Colab notebook (Swara_Pande_B1_B2_19_SI_LAB_PROJECT.ipynb). The code produces the results and figures shown in Section 6. The data file daily_temperature_patterns.csv must be in the working directory.

```
# ===== Cell 0: Imports =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import norm

# ===== Cell 1: Load dataset =====
df = pd.read_csv("daily_temperature_patterns.csv")

# ===== Cell 4: Shape and missing values =====
df.shape

# ===== Cell 5 =====
df.isnull().sum()

# ===== Cell 6: Data type conversion and cleaning =====
# Convert the required columns to the correct data types
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')

numeric_columns = [
    'Temperature',
    'Feels_Like_Temperature',
    'Humidity',
    'Rainfall_mm',
    'Wind_Speed_kmph',
    'Pressure_hPa',
    'Cloud_Cover_Percent',
    'Sunshine_Hours',
    'Temperature_Change'
]
```

```
df[numeric_columns] = df[numeric_columns].apply(
    pd.to_numeric, errors='coerce'
)

# Remove rows with missing Date or Temperature
df = df.dropna(subset=['Date', 'Temperature']).reset_index(drop=True)

# Define temperature for all your later analysis
temperature = df['Temperature']

# ===== Cell 7: Descriptive statistics =====
print("\nDESCRIPTIVE STATISTICS")

print(temperature.describe())

population_mean = temperature.mean()
population_median = temperature.median()
population_variance = temperature.var()
population_std = temperature.std()
population_skewness = temperature.skew()

print("\nPopulation Mean:", population_mean)
print("Population Median:", population_median)
print("Population Variance:", population_variance)
print("Population Standard Deviation:", population_std)
print("Population Skewness:", population_skewness)

# ===== Cell 8: Distribution plots =====
plt.figure(figsize=(8, 5))

sns.histplot(
    temperature,
    kde=True
)

plt.title("Distribution of Daily Temperature")
plt.xlabel("Temperature")
plt.ylabel("Frequency")

plt.show()

# ===== Cell 9 =====
plt.figure(figsize=(8, 4))

sns.boxplot(
    x=temperature
)

plt.title("Box Plot of Daily Temperature")
plt.xlabel("Temperature")

plt.show()

# ===== Cell 10: Comparison with Normal, Uniform and Binomial =====
normal_data = np.random.normal(
    population_mean,
    population_std,
    2000
)

plt.figure(figsize=(8, 5))

sns.histplot(
    temperature,
    kde=True,
    stat="density",
    label="Actual Temperature"
)

sns.kdeplot(
    normal_data,
    label="Normal Distribution"
)

plt.title("Temperature vs Normal Distribution")
plt.xlabel("Temperature")
```

```
plt.ylabel("Density")
plt.legend()

plt.show()

# ===== Cell 11 =====
uniform_data = np.random.uniform(
    temperature.min(),
    temperature.max(),
    2000
)

plt.figure(figsize=(8, 5))

sns.histplot(
    temperature,
    kde=True,
    stat="density",
    label="Actual Temperature"
)

sns.kdeplot(
    uniform_data,
    label="Uniform Distribution"
)

plt.title("Temperature vs Uniform Distribution")
plt.xlabel("Temperature")
plt.ylabel("Density")
plt.legend()

plt.show()

# ===== Cell 12 =====
binomial_data = np.random.binomial(
    n=20,
    p=0.5,
    size=2000
)

plt.figure(figsize=(8, 5))

sns.histplot(
    binomial_data,
    discrete=True
)

plt.title("Simulated Binomial Distribution")
plt.xlabel("Number of Successes")
plt.ylabel("Frequency")

plt.show()

# ===== Cell 13: Skewness =====
print("\nSkewness:", population_skewness)

if abs(population_skewness) < 0.5:
    print("Temperature distribution is approximately symmetric.")
elif population_skewness > 0:
    print("Temperature distribution is positively skewed.")
else:
    print("Temperature distribution is negatively skewed.")

# ===== Cell 14: Sampling methods =====
simple_sample = df.sample(
    n=30,
    random_state=42
)

print("\nSIMPLE RANDOM SAMPLING")

print("Mean:",
      simple_sample['Temperature'].mean())

print("Variance:",
      simple_sample['Temperature'].var())
```

```
print("Standard Deviation:",
      simple_sample['Temperature'].std())

# ===== Cell 15 =====
step = max(1, len(df) // 30)

start = np.random.randint(0, step)

systematic_sample = df.iloc[
    start::step
].head(30)

print("\nSYSTEMATIC SAMPLING")

print("Mean:",
      systematic_sample['Temperature'].mean())

print("Variance:",
      systematic_sample['Temperature'].var())

print("Standard Deviation:",
      systematic_sample['Temperature'].std())

# ===== Cell 16 =====
def get_season(month):

    if month in [12, 1, 2]:
        return "Winter"

    elif month in [3, 4, 5]:
        return "Summer"

    elif month in [6, 7, 8, 9]:
        return "Monsoon"

    else:
        return "Post-Monsoon"


df['Season_Calculated'] = df['Date'].dt.month.apply(
    get_season
)

# Take proportional samples from each season
stratified_parts = []

total_sample_size = 30

for season_name, group in df.groupby('Season_Calculated'):

    proportion = len(group) / len(df)

    sample_n = max(
        1,
        round(proportion * total_sample_size)
    )

    sample_n = min(
        sample_n,
        len(group)
    )

    part = group.sample(
        n=sample_n,
        random_state=42
    )

    stratified_parts.append(part)

stratified_sample = pd.concat(
    stratified_parts
)

# Adjust to exactly 30 if necessary
if len(stratified_sample) > 30:
    stratified_sample = stratified_sample.sample(
        n=30,
```

```
        random_state=42
    )

print("\nSTRATIFIED SAMPLING")

print("Mean:",
      stratified_sample['Temperature'].mean())

print("Variance:",
      stratified_sample['Temperature'].var())

print("Standard Deviation:",
      stratified_sample['Temperature'].std())

# ===== Cell 17 =====
df['Month_Calculated'] = df['Date'].dt.month

available_months = df['Month_Calculated'].unique()

selected_months = np.random.choice(
    available_months,
    size=3,
    replace=False
)

cluster_sample = df[
    df['Month_Calculated'].isin(selected_months)
]

print("\nCLUSTER SAMPLING")

print("Selected Months:",
      selected_months)

print("Number of Records:",
      len(cluster_sample))

print("Mean:",
      cluster_sample['Temperature'].mean())

print("Variance:",
      cluster_sample['Temperature'].var())

print("Standard Deviation:",
      cluster_sample['Temperature'].std())

# ===== Cell 18: Sampling comparison =====
sampling_results = pd.DataFrame({

    'Sampling Method': [
        'Simple Random',
        'Systematic',
        'Stratified',
        'Cluster'
    ],

    'Mean Temperature': [
        simple_sample['Temperature'].mean(),
        systematic_sample['Temperature'].mean(),
        stratified_sample['Temperature'].mean(),
        cluster_sample['Temperature'].mean()
    ],

    'Variance': [
        simple_sample['Temperature'].var(),
        systematic_sample['Temperature'].var(),
        stratified_sample['Temperature'].var(),
        cluster_sample['Temperature'].var()
    ],

    'Standard Deviation': [
        simple_sample['Temperature'].std(),
        systematic_sample['Temperature'].std(),
        stratified_sample['Temperature'].std(),
        cluster_sample['Temperature'].std()
    ]
})
```

```
print("\nSAMPLING METHODS COMPARISON")
print(sampling_results)

# ===== Cell 19: Sample statistics =====
comparison_results = []

for n in [30, 50, 100]:

    sample = temperature.sample(
        n=n,
        random_state=42
    )

    sample_mean = sample.mean()
    sample_median = sample.median()
    sample_variance = sample.var()
    sample_std = sample.std()

    comparison_results.append({

        'Sample Size': n,

        'Sample Mean': sample_mean,

        'Sample Median': sample_median,

        'Sample Variance': sample_variance,

        'Sample SD': sample_std
    })

comparison_table = pd.DataFrame(
    comparison_results
)

print("\nSAMPLE STATISTICS")
print(comparison_table)

# ===== Cell 20 =====
population_sample_table = pd.DataFrame({

    'Statistic': [
        'Mean',
        'Median',
        'Variance',
        'Standard Deviation'
    ],

    'Population': [
        population_mean,
        population_median,
        population_variance,
        population_std
    ],

    'Sample 30': [
        temperature.sample(30, random_state=42).mean(),
        temperature.sample(30, random_state=42).median(),
        temperature.sample(30, random_state=42).var(),
        temperature.sample(30, random_state=42).std()
    ],

    'Sample 50': [
        temperature.sample(50, random_state=42).mean(),
        temperature.sample(50, random_state=42).median(),
        temperature.sample(50, random_state=42).var(),
        temperature.sample(50, random_state=42).std()
    ],

    'Sample 100': [
        temperature.sample(100, random_state=42).mean(),
        temperature.sample(100, random_state=42).median(),
        temperature.sample(100, random_state=42).var(),
        temperature.sample(100, random_state=42).std()
    ]
})
```

```
print("\nPOPULATION VS SAMPLE")
print(population_sample_table)

# ===== Cell 21: Central Limit Theorem =====
print("\n\n")
print("=" * 60)
print("CENTRAL LIMIT THEOREM")
print("=" * 60)


clt_results = []

for n in [30, 50, 100, 200]:

    means = []

    for i in range(2000):

        sample = np.random.choice(
            temperature,
            size=n,
            replace=True
        )

        means.append(
            sample.mean()
        )

    means = np.array(means)

    theoretical_se = (
        population_std / np.sqrt(n)
    )

    observed_se = means.std(ddof=1)

    clt_results.append({

        'Sample Size': n,

        'Mean of Sample Means':
            means.mean(),

        'Theoretical SE':
            theoretical_se,

        'Observed SE':
            observed_se
    })

    # Separate plot for every sample size
    plt.figure(figsize=(8, 5))

    sns.histplot(
        means,
        kde=True
    )

    plt.axvline(
        population_mean,
        linestyle='--',
        label='Population Mean'
    )

    plt.title(
        f"CLT Sampling Distribution (n={n})"
    )

    plt.xlabel("Sample Mean Temperature")
    plt.ylabel("Frequency")

    plt.legend()

    plt.show()

    clt_table = pd.DataFrame(
```

```
    clt_results
)

print("\nCLT RESULTS")
print(clt_table)

# ===== Cell 22 =====
for n in [10, 30, 50, 100, 200]:

    se = population_std / np.sqrt(n)

    print(
        "Sample Size:",
        n,
        "| Standard Error:",
        se
    )

# ===== Cell 23: Bootstrap =====
bootstrap_means = {}

for n in [30, 50, 100]:
    sample = df['Temperature'].dropna().sample(
        n=n, random_state=42
    ).to_numpy()

    means = []
    for i in range(5000):
        bootstrap_sample = np.random.choice(
            sample, size=n, replace=True
        )
        means.append(bootstrap_sample.mean())

    bootstrap_means[n] = np.array(means)

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

for ax, n in zip(axes, [30, 50, 100]):
    sns.histplot(bootstrap_means[n], kde=True, ax=ax)

    ax.axvline(
        df['Temperature'].mean(),
        linestyle='--',
        label='Population Mean'
    )
    ax.axvline(
        df['Temperature'].dropna().sample(
            n=n, random_state=42
        ).mean(),
        linestyle=':',
        label='Sample Mean'
    )

    ax.set_title(f'Bootstrap Distribution (n={n})')
    ax.set_xlabel('Bootstrap Mean')
    ax.set_ylabel('Frequency')
    ax.legend(fontsize=8)

plt.tight_layout()
plt.show()

# ===== Cell 24 =====
bootstrap_results = []

for n in [30, 50, 100]:

    sample = temperature.sample(
        n=n,
        random_state=42
    )

    bootstrap_values = []

    for i in range(5000):

        b_sample = np.random.choice(
            sample,
```

```
            size=n,
            replace=True
        )

        bootstrap_values.append(
            b_sample.mean()
        )

    bootstrap_results.append({

        'Sample Size': n,

        'Sample Mean': sample.mean(),

        'Bootstrap Mean':
            np.mean(bootstrap_values),

        'Population Mean':
            population_mean
    })

bootstrap_table = pd.DataFrame(
    bootstrap_results
)

print("\nBOOTSTRAP RESULTS")
print(bootstrap_table)

# ===== Cell 25: Estimators: mean versus median =====
sample_means = []
sample_medians = []

for i in range(1000):

    sample = np.random.choice(
        temperature,
        size=30,
        replace=True
    )

    sample_means.append(
        sample.mean()
    )

    sample_medians.append(
        np.median(sample)
    )

# ===== Cell 26 =====
plt.figure(figsize=(8, 5))

sns.histplot(
    sample_means,
    kde=True
)

plt.axvline(
    population_mean,
    linestyle='--',
    label='Population Mean'
)

plt.title(
    "Sampling Distribution of Sample Means"
)

plt.xlabel("Sample Mean")
plt.ylabel("Frequency")

plt.legend()

plt.show()

# ===== Cell 27 =====
plt.figure(figsize=(8, 5))

sns.histplot(
```

```
    sample_medians,
    kde=True
)

plt.axvline(
    population_median,
    linestyle='--',
    label='Population Median'
)

plt.title(
    "Sampling Distribution of Sample Medians"
)

plt.xlabel("Sample Median")
plt.ylabel("Frequency")

plt.legend()

plt.show()

# ===== Cell 28 =====
mean_estimator_variance = np.var(
    sample_means,
    ddof=1
)

median_estimator_variance = np.var(
    sample_medians,
    ddof=1
)

print("\nVariance of Mean Estimator:",
      mean_estimator_variance)

print("Variance of Median Estimator:",
      median_estimator_variance)
estimator_results = []

for n in [30, 50, 100]:

    means = []
    medians = []

    for i in range(1000):

        sample = np.random.choice(
            temperature,
            size=n,
            replace=True
        )

        means.append(
            sample.mean()
        )

        medians.append(
            np.median(sample)
        )

    estimator_results.append({

        'Sample Size': n,

        'Mean Estimator Variance':
            np.var(means, ddof=1),

        'Median Estimator Variance':
            np.var(medians, ddof=1),

        'Mean Estimator Average':
            np.mean(means),

        'Median Estimator Average':
            np.mean(medians)
    })
```

```
estimator_table = pd.DataFrame(
    estimator_results
)

print("\nMEAN VS MEDIAN RESULTS")
print(estimator_table)

# ===== Cell 29: Confidence intervals and coverage =====
confidence_results = []

for n in [10, 30, 50, 100]:

    sample = temperature.sample(
        n=n,
        random_state=42
    )

    sample_mean = sample.mean()

    sample_se = (
        sample.std(ddof=1)
        / np.sqrt(n)
    )

    z_value = norm.ppf(0.975)

    margin = z_value * sample_se

    lower = sample_mean - margin
    upper = sample_mean + margin

    confidence_results.append({

        'Sample Size': n,

        'Sample Mean': sample_mean,

        'Standard Error': sample_se,

        'Lower CI': lower,

        'Upper CI': upper
    })

confidence_table = pd.DataFrame(
    confidence_results
)

print(confidence_table)

# ===== Cell 30 =====
coverage_results = []

for n in [10, 30, 50, 100]:

    covered = 0

    for i in range(1000):

        sample = np.random.choice(
            temperature,
            size=n,
            replace=True
        )

        sample_mean = sample.mean()

        sample_se = (
            sample.std(ddof=1)
            / np.sqrt(n)
        )

        margin = (
            norm.ppf(0.975)
            * sample_se
        )
```

```
        lower = sample_mean - margin
        upper = sample_mean + margin

        if (
            lower <= population_mean
            and population_mean <= upper
        ):
            covered += 1

    coverage = covered / 1000

    coverage_results.append({

        'Sample Size': n,

        'Coverage Probability':
            coverage
    })

coverage_table = pd.DataFrame(
    coverage_results
)

print(coverage_table)

# ===== Cell 31: Trend and relationship plots =====
plt.figure(figsize=(12, 5))

plt.plot(
    df['Date'],
    df['Temperature']
)

plt.title(
    "Daily Temperature Pattern"
)

plt.xlabel("Date")
plt.ylabel("Temperature")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# ===== Cell 32 =====
df['Month_Number'] = df['Date'].dt.month

monthly_temperature = (
    df.groupby('Month_Number')['Temperature']
    .mean()
)

plt.figure(figsize=(8, 5))

plt.plot(
    monthly_temperature.index,
    monthly_temperature.values,
    marker='o'
)

plt.title(
    "Average Monthly Temperature"
)

plt.xlabel("Month")
plt.ylabel("Average Temperature")

plt.xticks(range(1, 13))

plt.show()

# ===== Cell 33 =====
seasonal_temperature = (
    df.groupby('Season_Calculated')['Temperature']
    .mean()
```

```
    .sort_values()
)

plt.figure(figsize=(8, 5))

plt.bar(
    seasonal_temperature.index,
    seasonal_temperature.values
)

plt.title(
    "Average Temperature by Season"
)

plt.xlabel("Season")
plt.ylabel("Average Temperature")

plt.show()

# ===== Cell 34 =====
weather_columns = [
    'Temperature',
    'Feels_Like_Temperature',
    'Humidity',
    'Rainfall_mm',
    'Wind_Speed_kmph',
    'Pressure_hPa',
    'Cloud_Cover_Percent',
    'Sunshine_Hours'
]

correlation = df[weather_columns].corr()

plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f"
)

plt.title(
    "Correlation Matrix of Weather Variables"
)

plt.show()

# ===== Cell 35 =====
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x='Humidity',
    y='Temperature'
)

plt.title(
    "Temperature vs Humidity"
)

plt.xlabel("Humidity")
plt.ylabel("Temperature")

plt.show()

# ===== Cell 36 =====
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x='Rainfall_mm',
    y='Temperature'
)

plt.title(
    "Temperature vs Rainfall"
)
```

```
plt.xlabel("Rainfall (mm)")
plt.ylabel("Temperature")

plt.show()

# ===== Cell 37: Hypothesis tests =====
#  Z-test and one-sample t-test

from scipy.stats import norm, ttest_1samp
import numpy as np

alpha = 0.05
mu0 = 27.0

x = df["Temperature"].dropna()
n = len(x)
sample_mean = x.mean()
sample_sd = x.std(ddof=1)

# One-sample t-test: population SD is unknown
t_stat, t_pvalue = ttest_1samp(x, popmean=mu0)

print("ONE-SAMPLE T-TEST")
print("Sample mean:", sample_mean)
print("t-statistic:", t_stat)
print("p-value:", t_pvalue)
print("Decision:", "Reject H0" if t_pvalue < alpha
      else "Fail to reject H0")

# Z-test demonstration:
# This assumes a genuinely known population SD.
# Here population_std is the SD of the available dataset,
# so the result is illustrative rather than a true known-SD Z-test.
sigma_known = population_std

z_stat = (sample_mean - mu0) / (sigma_known / np.sqrt(n))
z_pvalue = 2 * norm.sf(abs(z_stat))

print("\nONE-SAMPLE Z-TEST (illustrative)")
print("Z-statistic:", z_stat)
print("p-value:", z_pvalue)
print("Decision:", "Reject H0" if z_pvalue < alpha
      else "Fail to reject H0")

# ===== Cell 38 =====
#  Chi-square test of independence

from scipy.stats import chi2_contingency

contingency_table = pd.crosstab(df["Season"], df["Day_Type"])

print("Observed frequencies:")
display(contingency_table)

chi2_stat, p_value, dof, expected = chi2_contingency(
    contingency_table
)

print("Chi-square statistic:", chi2_stat)
print("Degrees of freedom:", dof)
print("p-value:", p_value)
print("Decision:", "Reject H0" if p_value < 0.05
      else "Fail to reject H0")

print("\nExpected frequencies:")
display(pd.DataFrame(
    expected,
    index=contingency_table.index,
    columns=contingency_table.columns
).round(2))

if (expected < 5).any():
    print("Warning: Some expected frequencies are below 5.")

# ===== Cell 39 =====
# Covariance analysis
```

```
weather_columns = [
    "Temperature",
    "Feels_Like_Temperature",
    "Humidity",
    "Rainfall_mm",
    "Wind_Speed_kmph",
    "Pressure_hPa",
    "Cloud_Cover_Percent",
    "Sunshine_Hours"
]

covariance_matrix = df[weather_columns].cov()

print("Covariance Matrix:")
display(covariance_matrix.round(3))

print("\nTemperature-Humidity Covariance:",
      df["Temperature"].cov(df["Humidity"]))

plt.figure(figsize=(10, 7))
sns.heatmap(covariance_matrix, annot=True, fmt=".1f",
            cmap="coolwarm", center=0)
plt.title("Covariance Matrix of Weather Variables")
plt.tight_layout()
plt.show()

# ===== Cell 40: Correlation tests =====
#  Pearson and Spearman hypothesis testing

from scipy.stats import pearsonr, spearmanr

pairs = [
    ("Temperature", "Humidity"),
    ("Temperature", "Rainfall_mm"),
    ("Temperature", "Sunshine_Hours"),
    ("Temperature", "Wind_Speed_kmph")
]

correlation_results = []

for col1, col2 in pairs:
    pair_data = df[[col1, col2]].dropna()

    pearson_r, pearson_p = pearsonr(
        pair_data[col1], pair_data[col2]
    )
    spearman_r, spearman_p = spearmanr(
        pair_data[col1], pair_data[col2]
    )

    correlation_results.append({
        "Variable 1": col1,
        "Variable 2": col2,
        "Pearson r": pearson_r,
        "Pearson p-value": pearson_p,
        "Pearson decision": (
            "Reject H0" if pearson_p < 0.05
            else "Fail to reject H0"
        ),
        "Spearman rho": spearman_r,
        "Spearman p-value": spearman_p,
        "Spearman decision": (
            "Reject H0" if spearman_p < 0.05
            else "Fail to reject H0"
        )
    })

correlation_test_table = pd.DataFrame(correlation_results)
display(correlation_test_table.round(4))

# ===== Cell 41: Two-way ANOVA =====
#  Two-way ANOVA

import statsmodels.api as sm
from statsmodels.formula.api import ols
```

```
anova_data = df[
    ["Temperature", "Season_Calculated", "Day_Type"]
].dropna().copy()

anova_model = ols(
    "Temperature ~ C(Season_Calculated) * C(Day_Type)",
    data=anova_data
).fit()

anova_table = sm.stats.anova_lm(anova_model, typ=2)

print("TWO-WAY ANOVA RESULTS")
display(anova_table)

for effect in anova_table.index:
    p = anova_table.loc[effect, "PR(>F)"]
    print(
        effect,
        "| p-value:", round(p, 5),
        "|", "Significant" if p < 0.05
        else "Not statistically significant"
    )

# Residual normality diagnostic
shapiro_stat, shapiro_p = stats.shapiro(anova_model.resid)
print("\nShapiro-Wilk residual normality test p-value:",
      shapiro_p)

# Equal-variance diagnostic across season/day-type groups
groups = [
    group["Temperature"].values
    for _, group in anova_data.groupby(
        ["Season_Calculated", "Day_Type"]
    )
    if len(group) >= 2
]

if len(groups) >= 2:
    levene_stat, levene_p = stats.levene(*groups)
    print("Levene test p-value:", levene_p)

# ===== Cell 42: Feature preparation and chronological split =====
#  Prepare features and split data chronologically

from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

ml_data = df.dropna(subset=["Date", "Temperature"]).copy()
ml_data["Date"] = pd.to_datetime(ml_data["Date"])
ml_data = ml_data.sort_values("Date").reset_index(drop=True)

target = "Temperature"

features = [
    "Humidity",
    "Rainfall_mm",
    "Wind_Speed_kmph",
    "Pressure_hPa",
    "Cloud_Cover_Percent",
    "Sunshine_Hours",
    "Season_Calculated",
    "Day_Type"
]

X = ml_data[features]
y = ml_data[target]

# First 80% for training, last 20% for testing
split_index = int(len(ml_data) * 0.8)

X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]
y_train = y.iloc[:split_index]
```

```
y_test = y.iloc[split_index:]

numeric_features = X.select_dtypes(include=np.number).columns.tolist()
categorical_features = X.select_dtypes(
    include=["object", "category", "string"]
).columns.tolist()

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
print("Features:", features)

# ===== Cell 43: Regression models =====
# Multiple linear regression

from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

linear_model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

linear_model.fit(X_train, y_train)
linear_predictions = linear_model.predict(X_test)

def evaluate_model(name, actual, predicted):
    return {
        "Model": name,
        "MAE": mean_absolute_error(actual, predicted),
        "RMSE": np.sqrt(mean_squared_error(actual, predicted)),
        "R2 Score": r2_score(actual, predicted)
    }

linear_results = evaluate_model(
    "Multiple Linear Regression",
    y_test,
    linear_predictions
)

display(pd.DataFrame([linear_results]).round(4))

plt.figure(figsize=(7, 5))
plt.scatter(y_test, linear_predictions)
low = min(y_test.min(), linear_predictions.min())
high = max(y_test.max(), linear_predictions.max())
plt.plot([low, high], [low, high], "r--")
plt.xlabel("Actual Temperature")
plt.ylabel("Predicted Temperature")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()

# ===== Cell 44 =====
# Ridge and Lasso regression

from sklearn.linear_model import Ridge, Lasso

ridge_model = Pipeline([
    ("preprocessor", preprocessor),
```

```
    ("regressor", Ridge(alpha=1.0))
])

lasso_model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", Lasso(alpha=0.1, max_iter=20000))
])

models = {
    "Ridge Regression": ridge_model,
    "Lasso Regression": lasso_model
}

regularization_results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    regularization_results.append(
        evaluate_model(name, y_test, predictions)
    )

display(pd.DataFrame(regularization_results).round(4))

# ===== Cell 45 =====
#  Decision Tree and Random Forest

from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

tree_model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", DecisionTreeRegressor(
        max_depth=5,
        random_state=42
    ))
])

forest_model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=200,
        max_depth=8,
        random_state=42,
        n_jobs=-1
    ))
])

tree_model.fit(X_train, y_train)
forest_model.fit(X_train, y_train)

tree_predictions = tree_model.predict(X_test)
forest_predictions = forest_model.predict(X_test)

tree_results = evaluate_model(
    "Decision Tree", y_test, tree_predictions
)
forest_results = evaluate_model(
    "Random Forest", y_test, forest_predictions
)

display(pd.DataFrame([
    tree_results, forest_results
]).round(4))

# ===== Cell 46: Model selection (time-series CV) =====
#Model selection using time-series cross-validation

from sklearn.model_selection import TimeSeriesSplit, GridSearchCV

candidate_models = {
    "Linear Regression": (
        Pipeline([
            ("preprocessor", preprocessor),
            ("regressor", LinearRegression())
        ]),
        {}
```

```
    ),

    "Ridge": (
        Pipeline([
            ("preprocessor", preprocessor),
            ("regressor", Ridge())
        ]),
        {"regressor__alpha": [0.1, 1, 10, 100]}
    ),

    "Lasso": (
        Pipeline([
            ("preprocessor", preprocessor),
            ("regressor", Lasso(max_iter=20000))
        ]),
        {"regressor__alpha": [0.01, 0.1, 1.0]}
    ),

    "Decision Tree": (
        Pipeline([
            ("preprocessor", preprocessor),
            ("regressor", DecisionTreeRegressor(random_state=42))
        ]),
        {
            "regressor__max_depth": [3, 5, 8],
            "regressor__min_samples_leaf": [2, 5]
        }
    ),

    "Random Forest": (
        Pipeline([
            ("preprocessor", preprocessor),
            ("regressor", RandomForestRegressor(
                n_estimators=100,
                random_state=42,
                n_jobs=1
            ))
        ]),
        {
            "regressor__max_depth": [4, 8, None],
            "regressor__min_samples_leaf": [2, 5]
        }
    )
}

tscv = TimeSeriesSplit(n_splits=5)
selection_results = []
best_model = None
best_model_name = None
best_cv_rmse = float("inf")

for name, (pipeline, parameters) in candidate_models.items():
    search = GridSearchCV(
        estimator=pipeline,
        param_grid=parameters,
        scoring="neg_root_mean_squared_error",
        cv=tscv,
        n_jobs=-1
    )

    search.fit(X_train, y_train)

    cv_rmse = -search.best_score_
    predictions = search.predict(X_test)

    selection_results.append({
        "Model": name,
        "CV RMSE": cv_rmse,
        "Test RMSE": np.sqrt(
            mean_squared_error(y_test, predictions)
        ),
        "Test MAE": mean_absolute_error(y_test, predictions),
        "Test R2": r2_score(y_test, predictions),
        "Best Parameters": search.best_params_
    })

    if cv_rmse < best_cv_rmse:
```

```
        best_cv_rmse = cv_rmse
        best_model = search.best_estimator_
        best_model_name = name

selection_table = pd.DataFrame(
    selection_results
).sort_values("CV RMSE")

display(selection_table.round(4))

print("Selected model:", best_model_name)
print("Best cross-validation RMSE:", round(best_cv_rmse, 4))

# ===== Cell 47: Save and load model =====
#Save and load the selected model

import joblib

model_path = "/content/best_temperature_model.joblib"

joblib.dump(best_model, model_path)

print("Model saved successfully:", model_path)

loaded_model = joblib.load(model_path)

# Verify that the saved model produces the same predictions
original_predictions = best_model.predict(X_test)
loaded_predictions = loaded_model.predict(X_test)

print(
    "Saved model verification:",
    np.allclose(original_predictions, loaded_predictions)
)

# Download the model from Colab
from google.colab import files
files.download(model_path)

# ===== Cell 48: GUI =====
import ipywidgets as widgets
from IPython.display import display, clear_output

numeric_inputs = {}
categorical_inputs = {}

for col in numeric_features:
    numeric_inputs[col] = widgets.FloatText(
        value=float(X_train[col].median()),
        description=col,
        style={"description_width": "initial"},
        layout=widgets.Layout(width="450px")
    )

for col in categorical_features:
    options = sorted(X_train[col].dropna().unique().tolist())

    categorical_inputs[col] = widgets.Dropdown(
        options=options,
        description=col,
        style={"description_width": "initial"},
        layout=widgets.Layout(width="450px")
    )

predict_button = widgets.Button(
    description="Predict Temperature",
    button_style="primary"
)

output = widgets.Output()

def predict_temperature(button):
    with output:
        clear_output(wait=True)

        values = {}
```

```
        for col, widget in numeric_inputs.items():
            values[col] = widget.value

        for col, widget in categorical_inputs.items():
            values[col] = widget.value

        new_data = pd.DataFrame([values], columns=features)
        prediction = loaded_model.predict(new_data)[0]

        print(f"Predicted Temperature: {prediction:.2f}")
        print("Units are the same as the Temperature column in your CSV.")

predict_button.on_click(predict_temperature)

display(widgets.VBox(
    list(numeric_inputs.values())
    + list(categorical_inputs.values())
    + [predict_button, output]
))
```

6. RESULTS AND DISCUSSION

This section presents the actual outputs of the notebook. Figures are numbered in order of appearance in the report; Figure 2 (the GUI layout) is in Section 4.3.

6.1 Required Output Screenshots

Data preprocessing output. After type conversion the data have 365 rows and 13 columns with no missing Date or Temperature values, and the chronological split gives 292 training and 73 testing records with the eight input features listed in Section 3.3. The exploratory plots are shown in Figure 1.

Distribution of temperature. Figure 3 compares the observed temperature with theoretical distributions. The Normal curve (same mean and standard deviation) matches the centre and overall spread of the data, consistent with the skewness of 0.10, but it does not reproduce the two humps of the observed histogram. The Uniform distribution over the same range is flat and also does not fit, since cooler and hotter days are more common than the extremes. The simulated Binomial is shown only as an example of a discrete distribution, which does not suit a continuous measurement such as temperature.

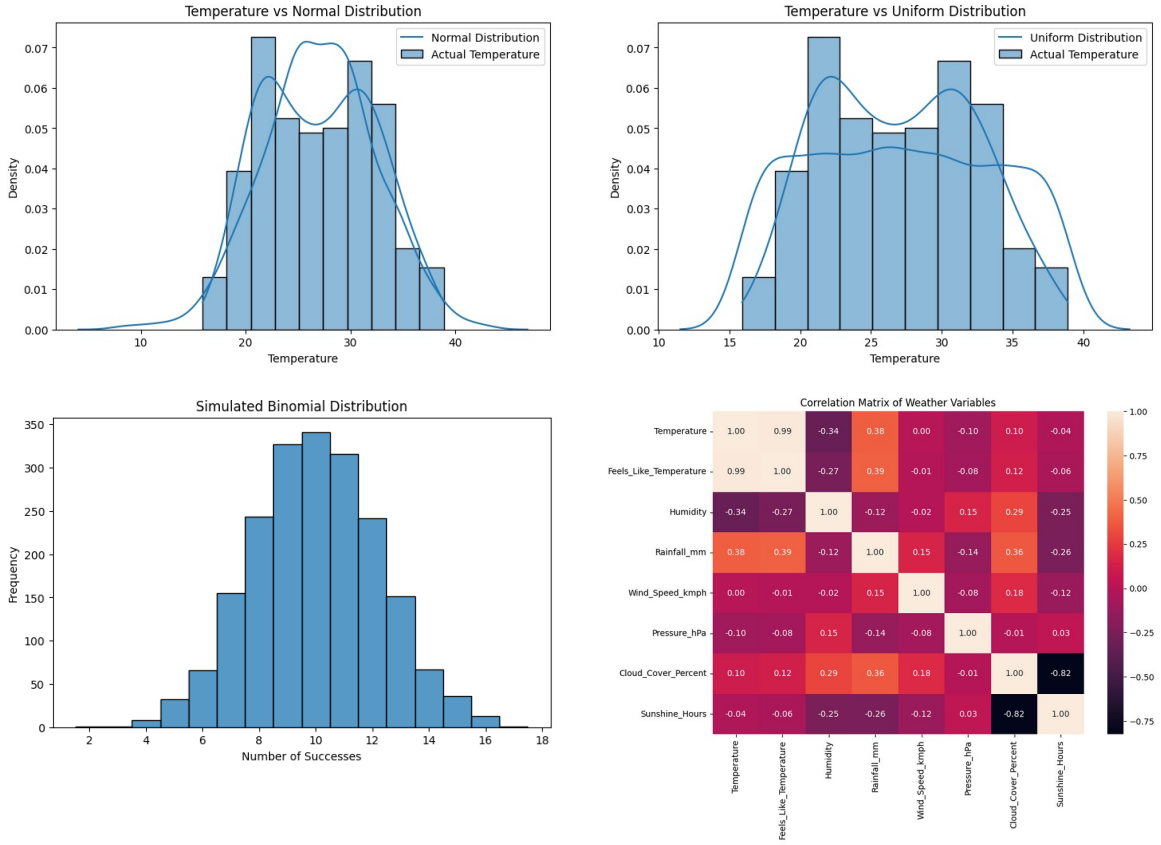


Figure 3: Temperature against a Normal distribution, against a Uniform distribution, a simulated Binomial(20, 0.5) distribution, and the correlation matrix of the weather variables.

Sampling methods and sample statistics

Samples of 30 days were drawn by four methods and compared with the population (mean 26.98, variance 28.35, SD 5.32). All four give means within about 0.5 °C of the population mean.

Table 4: Sampling methods compared (n = 30; the cluster sample used three whole months = 88 days)

Sampling method	Mean	Variance	Std. deviation
Simple random	26.893	33.152	5.758
Systematic	27.063	36.089	6.007
Stratified	27.483	29.345	5.417
Cluster	26.569	27.922	5.284

Table 5: Population versus simple random samples of different sizes

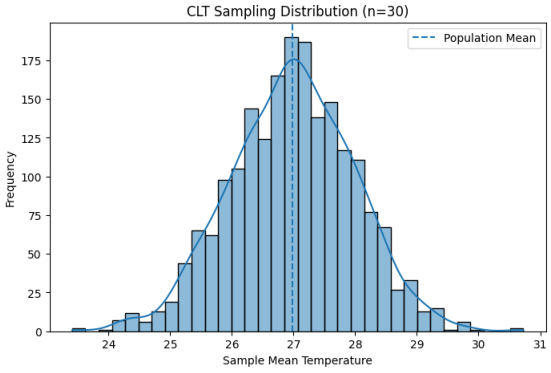
Statistic	Population	n = 30	n = 50	n = 100
Mean	26.984	26.893	25.688	26.160
Median	26.800	27.850	24.850	24.850
Variance	28.352	33.152	30.655	30.965
Std. deviation	5.325	5.758	5.537	5.565

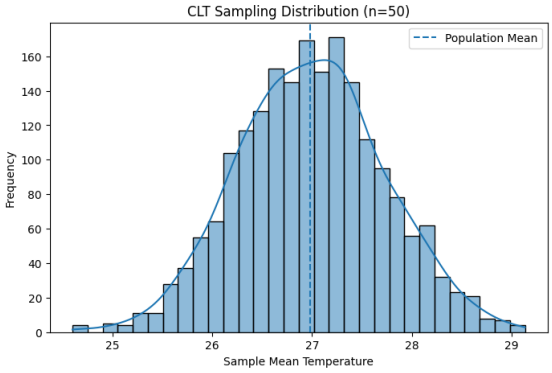
Central Limit Theorem, bootstrap and estimators

Means of 2,000 resamples were drawn for each sample size. The mean of the sample means stays at about 26.98 and the observed standard error follows the theoretical value σ/√n closely, falling from about 1.00 at n = 30 to 0.38 at n = 200 (Table 6, Figure 4). The sampling distribution looks increasingly normal and narrow as n grows.

Table 6: Central Limit Theorem results

Sample size	Mean of sample means	Theoretical SE	Observed SE
30	26.983	0.972	1.002
50	26.990	0.753	0.734
100	26.982	0.532	0.534
200	26.977	0.377	0.379





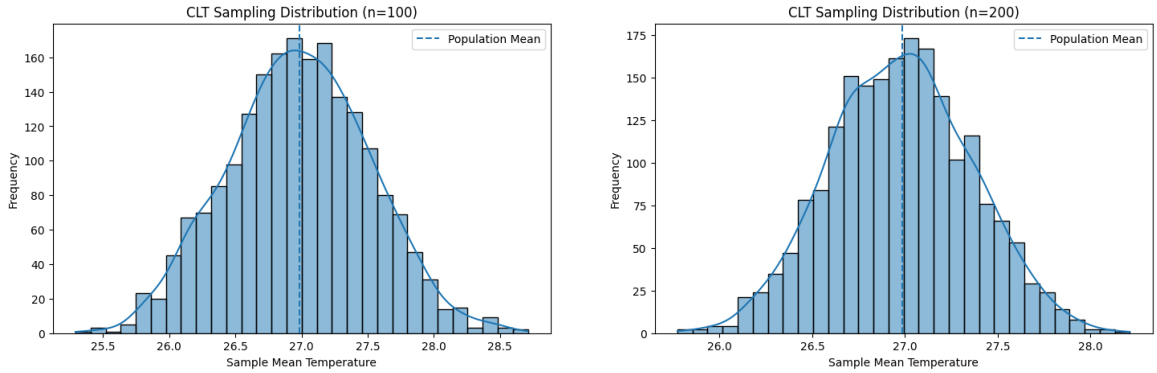


Figure 4: Sampling distribution of the mean for n = 30, 50, 100 and 200 (2,000 resamples each).

Bootstrap resampling (5,000 resamples) gives bootstrap means that are essentially equal to the sample means (for example 26.907 versus 26.893 at n = 30), showing that bootstrap estimates the sampling variability of a sample rather than correcting its bias; the sample mean itself can be about 1.3 °C away from the population mean (25.688 at n = 50).

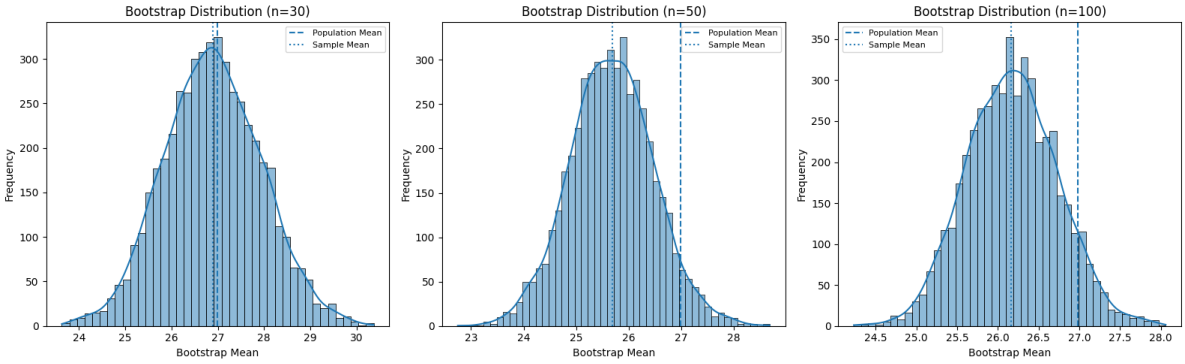
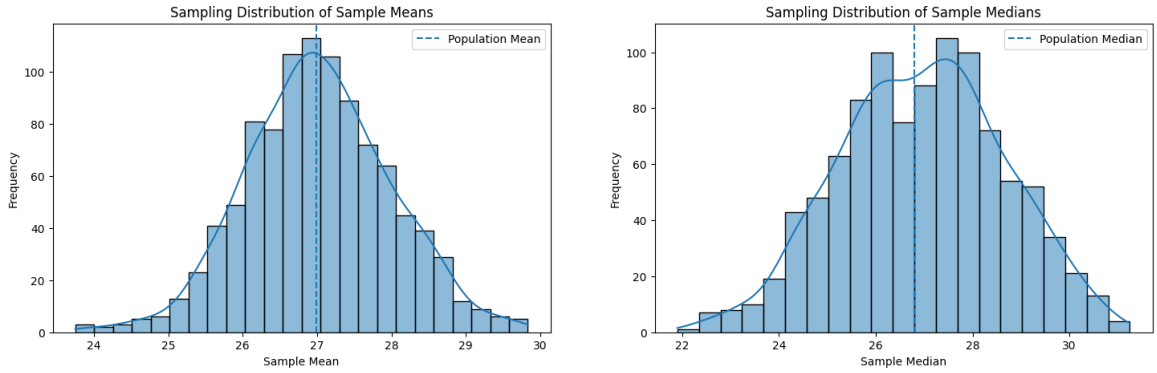


Figure 5: Bootstrap distributions of the mean for n = 30, 50 and 100.

Mean versus median as estimators: across 1,000 samples the mean has a much smaller variance than the median (0.94 versus 2.87 at n = 30, and 0.27 versus 0.93 at n = 100), so the mean is the more efficient estimator of the centre for this nearly symmetric data. Both are approximately unbiased (averages near 26.9–27.0).



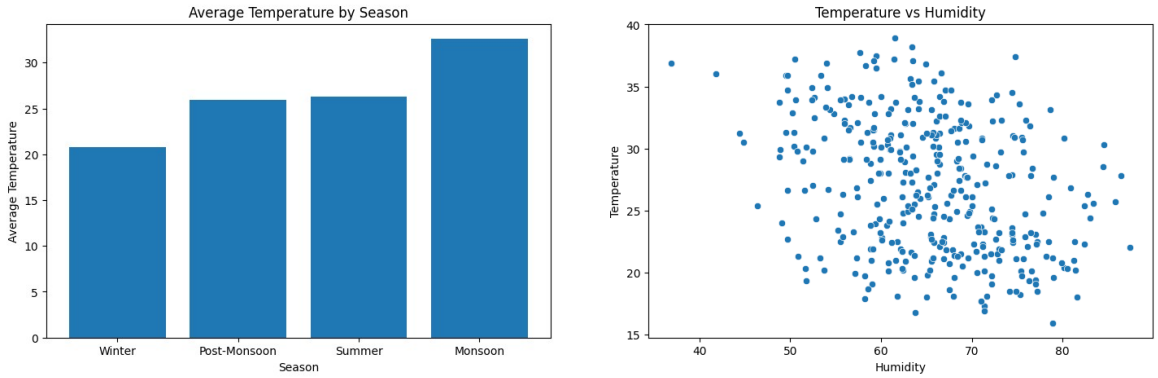


Figure 6: Sampling distribution of the sample mean, of the sample median, average temperature by season, and temperature versus humidity.

Confidence intervals

Table 7 gives 95% confidence intervals for the mean from simple random samples. The intervals shrink as the sample grows and all four intervals contain the population mean of 26.98. A simulation of 1,000 repeated samples gave empirical coverage of 92.0%, 93.3%, 93.5% and 95.3% for n = 10, 30, 50 and 100, approaching the nominal 95% as n grows. The slightly low coverage at small n arises because the normal (Z) critical value is used with an estimated standard deviation.

Table 7: 95% confidence intervals for the mean temperature, and simulated coverage

Sample size	Sample mean	Std. error	Lower CI	Upper CI	Coverage
10	27.360	2.096	23.252	31.468	0.920
30	26.893	1.051	24.833	28.954	0.933
50	25.688	0.783	24.153	27.223	0.935
100	26.160	0.556	25.069	27.251	0.953

Hypothesis tests, correlation and ANOVA

- **One-sample t-test (H0: mean = 27 °C):** t = −0.056, p = 0.955, fail to reject H0. An illustrative Z-test gives the same decision (z = −0.056, p = 0.955); it uses the dataset's own standard deviation, so it is a demonstration and not a true known-σ test.
- **Chi-square test of independence (Season and Day_Type):** χ^2 = 0.213, df = 3, p = 0.975, fail to reject H0. Weekday/weekend counts do not depend on the season, as expected. All expected frequencies are above 5.
- **Pearson and Spearman correlation with temperature:** see Table 8.
- **Two-way ANOVA (Season × Day_Type):** Season is highly significant (F = 310.33, p ≈ 4.4×10^{-99}); Day_Type is not significant (F = 1.78, p = 0.183); the interaction is not significant (F = 1.20, p = 0.311). Residuals pass the Shapiro–Wilk normality check (p = 0.146), but Levene's test (p ≈ 8.4×10^{-6}) shows unequal variances across groups, so the ANOVA p-values should be read with that caution; the season effect is so large that it is very unlikely to change.

Table 8: Correlation of temperature with other variables

Variable	Pearson r	p-value	Spearman ρ	p-value	Decision (α = 0.05)
Humidity	−0.337	< 0.001	−0.327	< 0.001	Significant
Rainfall_mm	0.385	< 0.001	0.433	< 0.001	Significant
Sunshine_Hours	−0.039	0.455	−0.032	0.541	Not significant
Wind_Speed_kmph	0.000	0.996	−0.004	0.940	Not significant

The covariance of temperature with humidity is −15.60, and the covariance matrix and the correlation matrix (Figure 3) show the same pattern as the tests. Temperature and feels-like temperature are almost identical (covariance 28.08 against a temperature variance of 28.35).

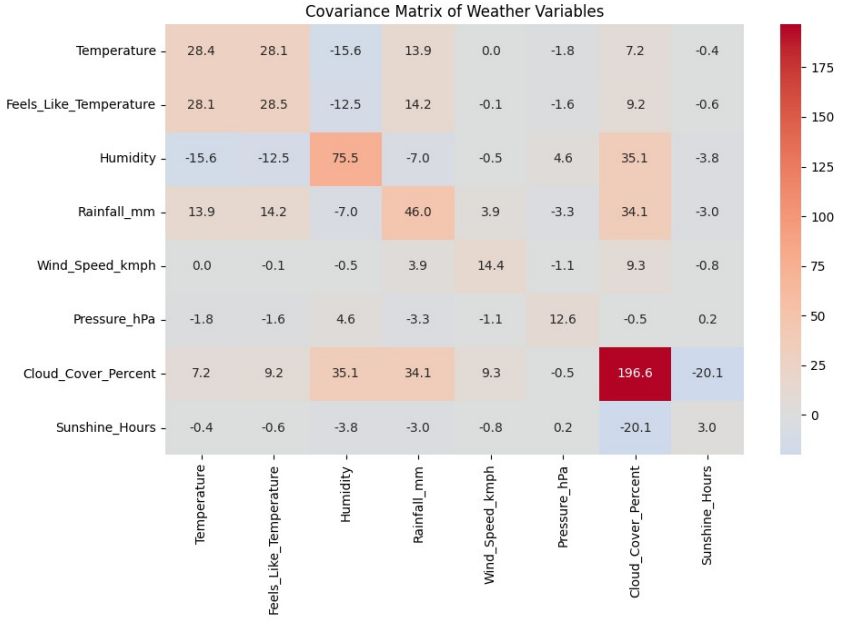
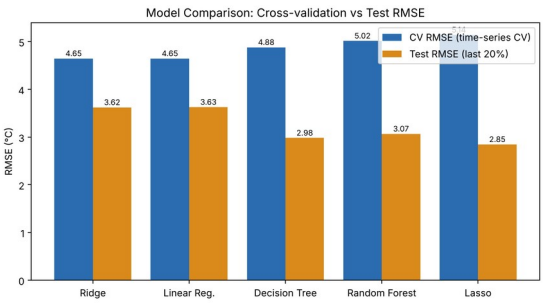
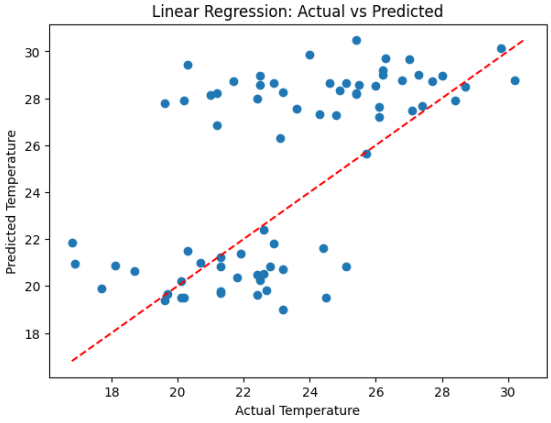


Figure 7: Covariance matrix of the weather variables.

Model results, performance and best model

Table 3 (Section 3.5) lists the performance of all models. Figure 8 compares cross-validation and test RMSE and plots actual against predicted temperature for the linear model on the test days (points on the dashed diagonal would be perfect predictions). For a regression problem this plot takes the place of the confusion matrix.





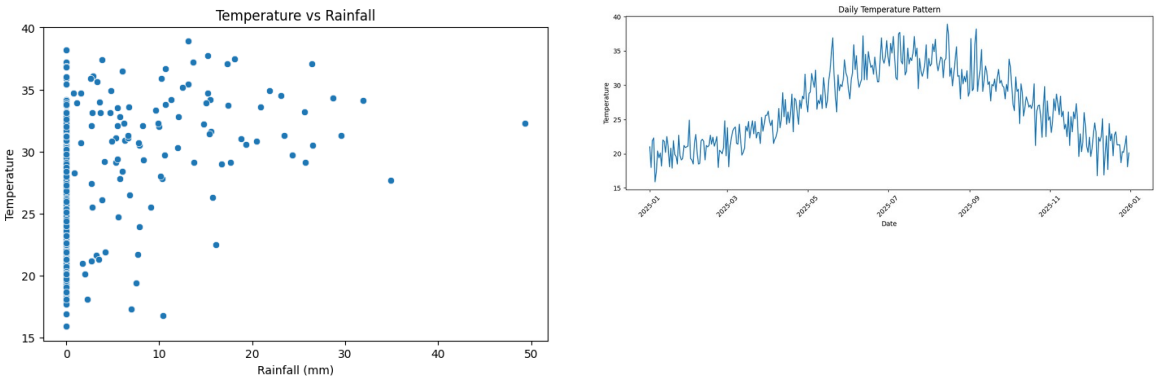


Figure 8: Model comparison (CV and test RMSE); actual versus predicted temperature for Linear Regression on the test days; temperature versus rainfall; daily temperature through the year.

Best model selection. Ridge Regression (alpha = 0.1) was selected with the lowest cross-validation RMSE of 4.6459 and a test RMSE of 3.6219 °C, test MAE of 2.8518 °C and R^2 of −0.4508. The saved model reproduced its predictions exactly after reloading. The GUI prediction result is shown in Figure 2.

6.2 Result Analysis

- **Temperature is symmetric, two-humped and driven by season.** The distribution is symmetric (skewness 0.10) with the same centre and spread as a Normal curve, but it has two humps and is not a close fit to a Normal or Uniform distribution. Monsoon days average 32.6 °C against 20.8 °C in Winter, and season explains most of the variation (ANOVA F = 310). Day type has no effect.
- **Sampling behaves as theory predicts.** Standard errors follow $\sigma/\sqrt{n}$, intervals narrow with sample size, coverage approaches 95%, and the mean is a much more efficient estimator than the median.
- **Relationships are moderate.** Temperature has a negative relationship with humidity and a positive one with rainfall (in this dataset the Monsoon season is also the hottest, so the rainfall link most likely reflects season rather than rain raising the temperature), and no significant link with wind or sunshine.
- **The regression models do not generalise well to the test period.** Test R^2 is below zero for Linear, Ridge and Random Forest and only 0.02–0.10 for Decision Tree and Lasso, with errors of about 2.2–2.9 °C (MAE) against a temperature standard deviation of 5.3 °C. The main reason is the chronological split: the training days contain only 19 Post-Monsoon days and the test days (20 Oct–31 Dec) are entirely Post-Monsoon (42 days) and Winter (31 days), so the models are partly asked to predict seasons they have barely seen. Winter appears in training with only 59 days.
- **Model ranking depends on how it is judged.** Ridge and Linear Regression are best in cross-validation, while Lasso and Decision Tree are best on the test period; the differences are within what a single 73-day test period can produce, and Lasso's coefficient shrinkage may help on unseen seasons. The test results are a single split and should be interpreted accordingly.
- **Objectives achieved.** The statistical analysis, model comparison, selection, saving and GUI were all completed. The predictive accuracy objective was only partly achieved: the

prediction is useful as a rough estimate but should not be relied on for precise temperatures.

Limitations and errors: one year of data, one location, a small training set (292 days), no information on the previous days' temperature, and a test period covering only two seasons. Linear models may also under-fit the strongly seasonal pattern.

7. CONCLUSION

This project analysed 365 days of 2025 weather data with the statistical methods of the SI Lab and built a regression model to predict daily temperature. The temperature is symmetric (mean 26.98 °C, SD 5.32 °C, skewness 0.10) though two-humped rather than a clean bell curve, depends strongly on season (Monsoon about 32.6 °C, Winter about 20.8 °C) and not on weekday or weekend, and is moderately related to humidity and rainfall. Experiments with four sampling methods, the Central Limit Theorem, bootstrapping and confidence intervals confirmed standard theory, including coverage approaching 95% and the greater efficiency of the mean over the median.

Five regression models were trained and tuned with time-series cross-validation. Ridge Regression (alpha = 0.1) was selected, saved with joblib and connected to an ipywidgets GUI. Its test performance (MAE 2.85 °C, RMSE 3.62 °C, R^2 −0.45) is limited, mainly because the chronological test period contains seasons that are rare in the training data. Possible improvements are to use several years of data, add lag features (previous days' temperatures) and month or day-of-year features, evaluate with rolling-origin validation across more seasons, and deploy the GUI as a standalone web application.

8. GITHUB LINK

GitHub Repository URL: **[Paste your repository URL here]**

The repository should contain the Python code files, the dataset (daily_temperature_patterns.csv), the trained model file (best_temperature_model.joblib), the GUI code and the Colab notebook.

9. REFERENCES

[1] Daily Temperature Patterns dataset, daily_temperature_patterns.csv (365 daily records, 2025), used as supplied for the SI Lab project.

[2] Scikit-learn documentation, https://scikit-learn.org/stable/

[3] pandas documentation, https://pandas.pydata.org/docs/

[4] SciPy documentation (scipy.stats), https://docs.scipy.org/doc/scipy/

[5] Statsmodels documentation, https://www.statsmodels.org/stable/

[6] Matplotlib documentation, https://matplotlib.org/stable/

[7] Seaborn documentation, https://seaborn.pydata.org/

[8] ipywidgets documentation, https://ipywidgets.readthedocs.io/

[9] Google Colaboratory, https://colab.research.google.com/